AJAIA

Ajaia Docs Submission report

AI-Native Full Stack Developer Assignment · lightweight collaborative document editor



Muhammad Armaghan Abbas

Full Stack Developer — candidate

✉ armaghanabbas11@gmail.com
☎ +92 325 9377334 (phone / WhatsApp)
GitHub: github.com/Muhammad-Armghan-Abbas
LinkedIn: linkedin.com/in/armghan-abbas

LIVE APP
ajaia-docs-gilt-xi.vercel.app

SOURCE CODE
github.com/Muhammad-Armghan-Abbas/Assignment-for-Full-Stack-Developer

WALKTHROUGH VIDEO
See VIDEO_URL.txt in the Drive folder

SIGN-IN
Mocked — pick a demo user, no password

DEMO USER	EMAIL	STARTING STATE
Alice Tan	alice@ajaia.test	Owns “Welcome to Ajaia Docs”; commenter on Bob's plan
Bob Lim	bob@ajaia.test	Owns “Q4 Launch Plan” (with a comment thread); editor on Alice's doc
Chris Ng	chris@ajaia.test	Viewer on the launch plan; use him to test sharing

3-minute check: sign in as Alice and import `sample-files/q1-pilot-report.docx` . Comment on a sentence, then share with Bob as Commenter. In a private window, sign in as Bob: Alice sees his avatar appear, and he can reply but not edit. As Alice, open Version history and restore.

Built with Next.js 16 · TipTap 3 · libSQL/Turso · Vercel

October 2026

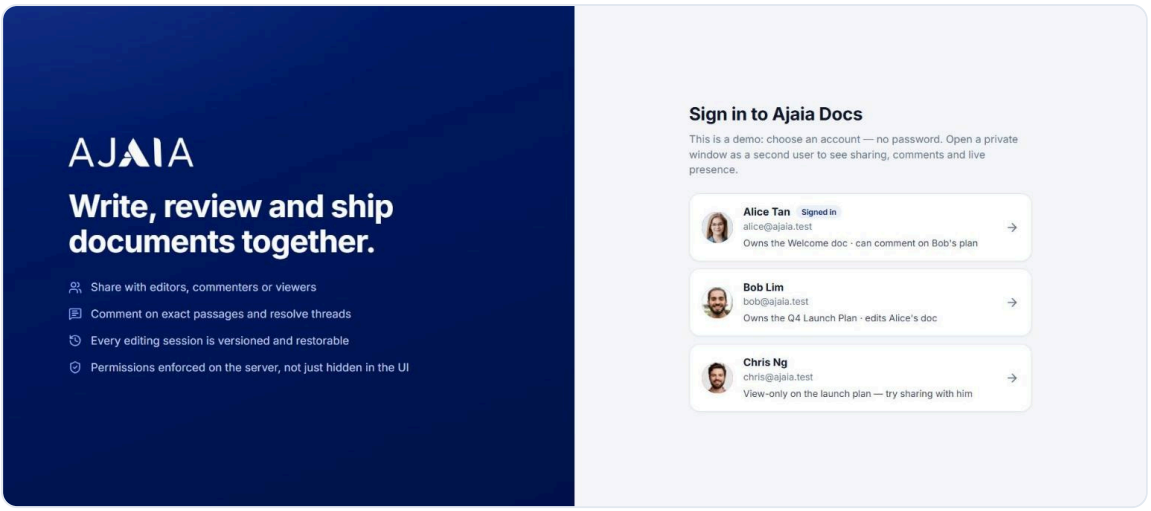
1 · Requirements coverage

Every core requirement is met end to end on the live deployment. All five optional stretch items were added, marked +. Each was built only after the core loop was complete and covered by tests. "S#" refers to the screenshots in section 2.

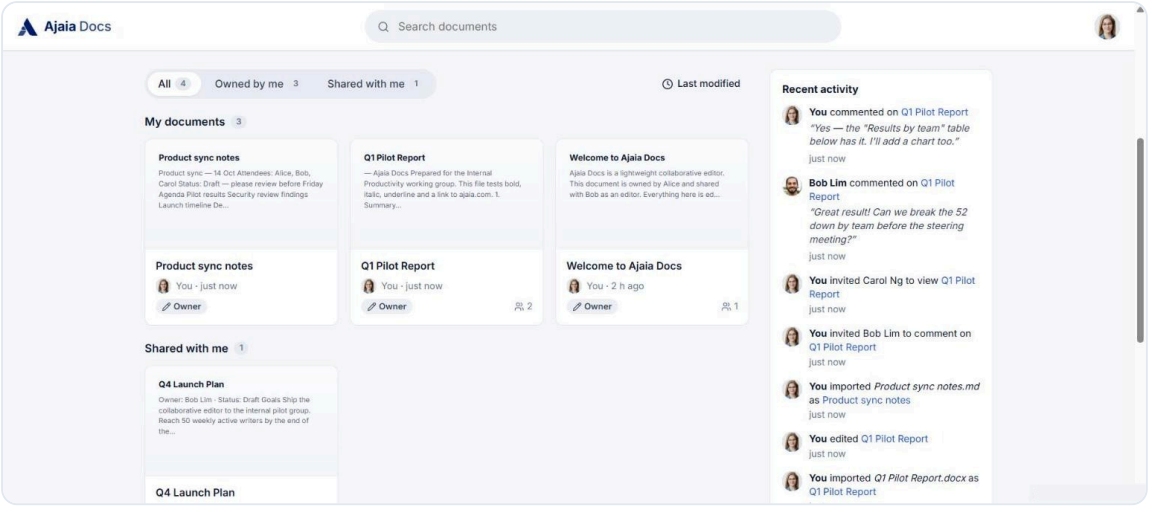
	REQUIREMENT	HOW IT'S DELIVERED	EVIDENCE
1. Create & edit			
✓	Create a new document	Blank, from 3 templates, or by import; title editable inline	S3
✓	Rename a document	Click the title; validated 1–200 chars; logged in activity	S3
✓	Edit in the browser	TipTap editor, autosave ≈1 s after typing, Ctrl+S, save status pill	S3, S5
✓	Save and reopen	Persisted in Turso; e2e test reloads and re-checks formatting	e2e #1
✓	Bold / italic / underline	Toolbar, selection bubble, keyboard shortcuts	S3
✓	Headings / text size	H1–H3 plus font-size menu	S3, S5
✓	Bulleted / numbered lists	Plus checklists	S3
2. File upload			
✓	.txt / .md / .docx → new document	Structure kept: headings, lists, tables, links; Word images become stored assets	S3, S4
✓	Limits stated in UI + README	".txt, .md and .docx up to 4 MB" under Import; clear error otherwise	S11
✓	Images in documents +	Upload, paste or drag-drop; resizable; access-checked URLs	S4
3. Sharing			
✓	A document owner	Ownership column; owner badge; only owner can share/delete	S6
✓	Grant another user access	Share by email as Editor / Commenter / Viewer; change or revoke	S6
✓	Owned vs shared distinction	"My documents" vs "Shared with me" sections + role badges on every card	S2, S9
4. Persistence			
✓	Survives refresh	libSQL: SQLite locally, Turso in production; forward-only migrations	e2e #1
✓	Formatting preserved	Sanitised HTML round-trip, tested per element type	unit
✓	Shared access demonstrable	3 seeded users with different roles on 2 seeded docs	S8, S9
5. Engineering quality			
✓	Setup / run instructions	README.md (local + Vercel/Turso deploy)	—
✓	Working deployment	https://ajaia-docs-gilt-xi.vercel.app	S1–S11
✓	Validation & error handling	Server-side validation on every input; one JSON error shape; 404 vs 403	S11
✓	Automated tests	31 Vitest (domain, permissions, import, migrations) + 6 multi-user Playwright	—
✓	Architecture note	ARCHITECTURE.md	—
Optional stretch (all five)			
✓	Real-time indicators +	Presence avatars + live updates via a 4 s sync heartbeat	e2e #1
✓	Commenting +	Text-anchored threads, replies, resolve/reopen, commenter role	S7, S8
✓	Version history +	Editing-session snapshots, preview, non-destructive restore	S10
✓	Export +	Markdown, HTML, Print → PDF	—
✓	Role-based permissions +	Owner / editor / commenter / viewer matrix, enforced server-side	S6

2 · The product, screen by screen

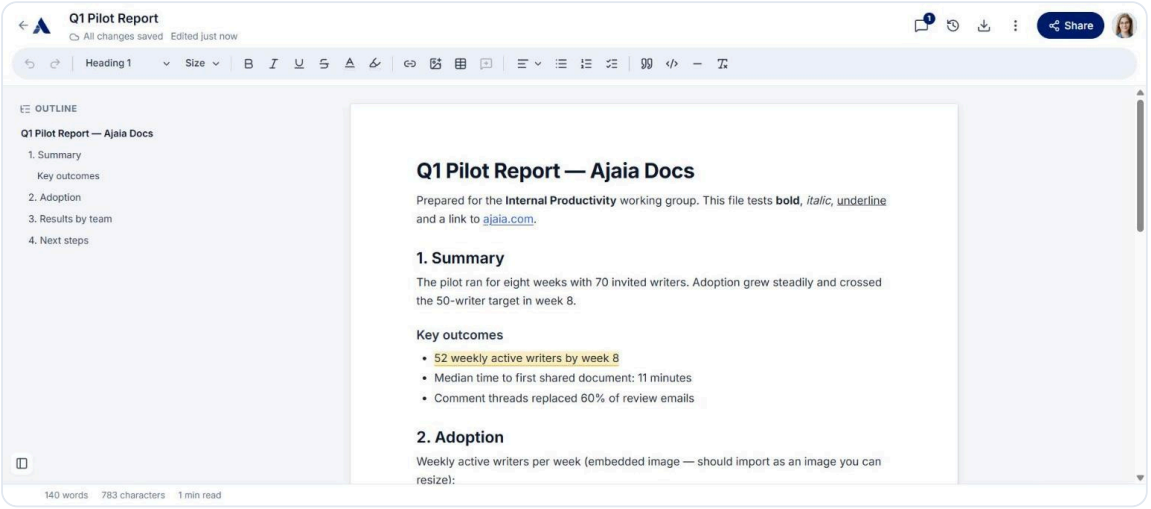
All screenshots were taken on the live deployment at ajaia-docs-gilt-xi.vercel.app with the sample files in `sample-files/`.



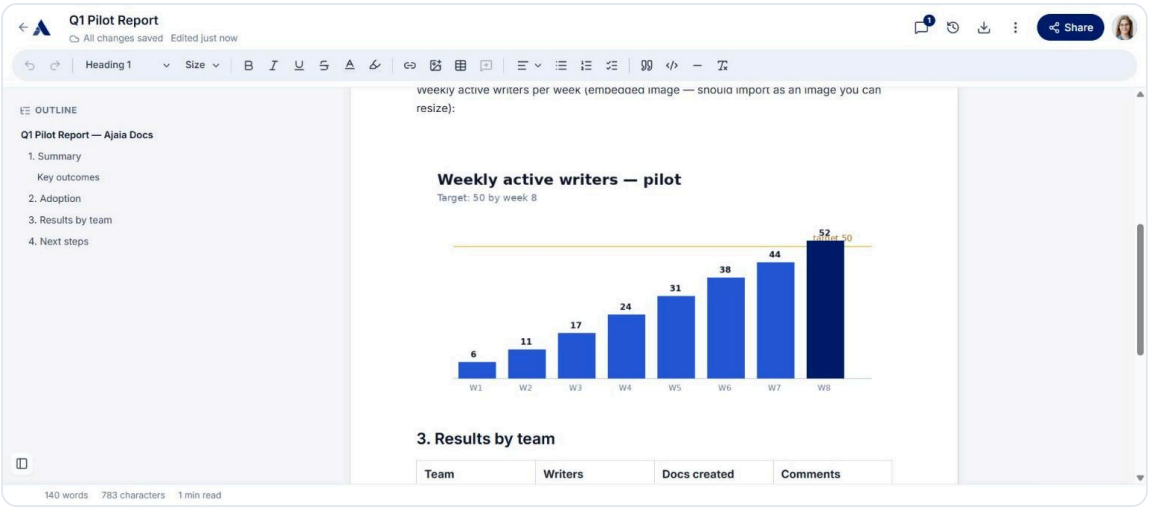
S1 · Sign-in — mocked auth with three seeded users and photo avatars; the session cookie is HMAC-signed



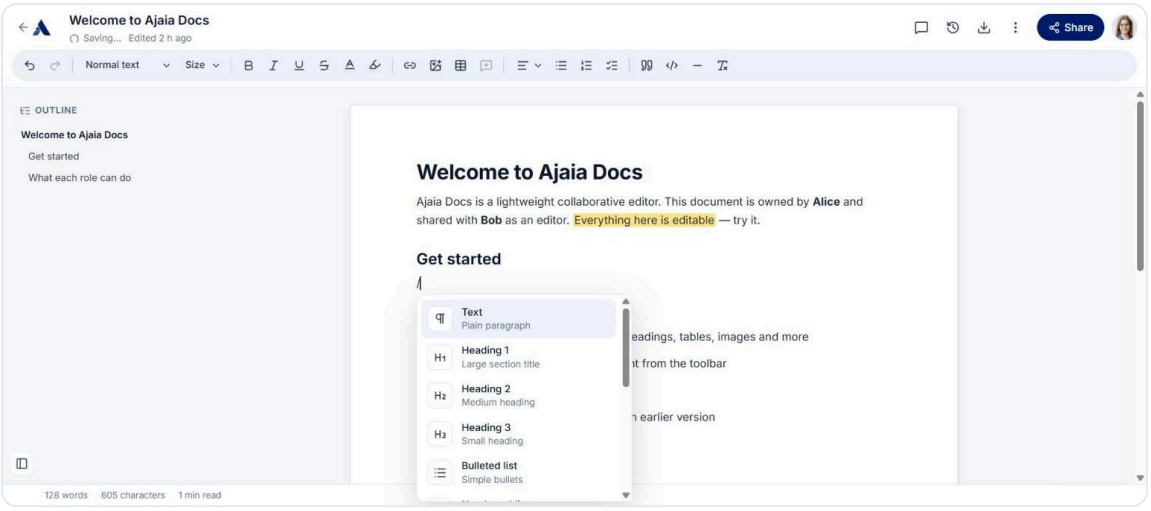
S2 · Documents home — owned vs shared sections, role badges, collaborator counts and the activity feed



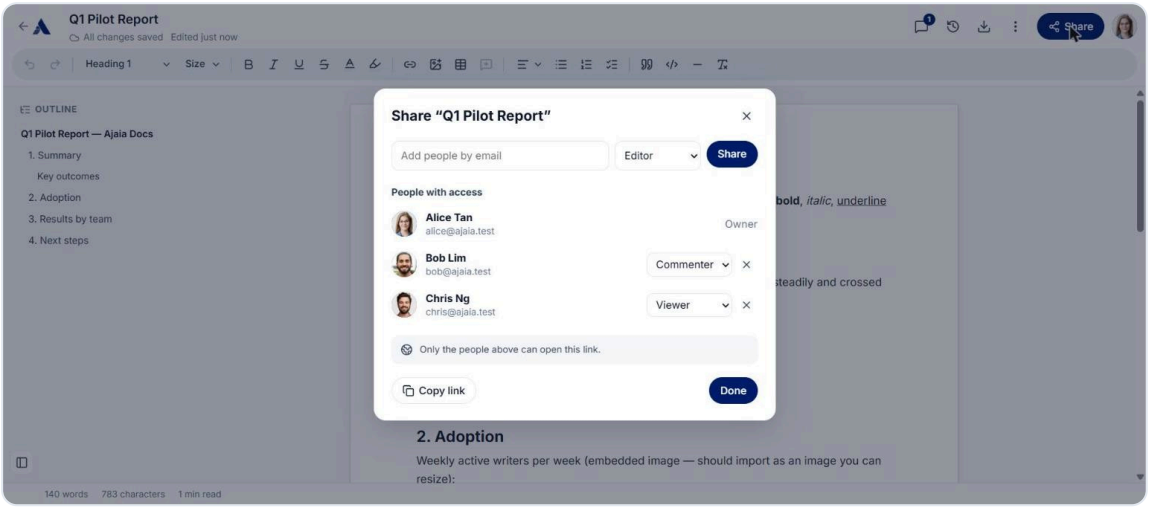
S3 · Editor (imported Word file) — headings, bold / italic / underline and a link preserved; toolbar, outline and word count



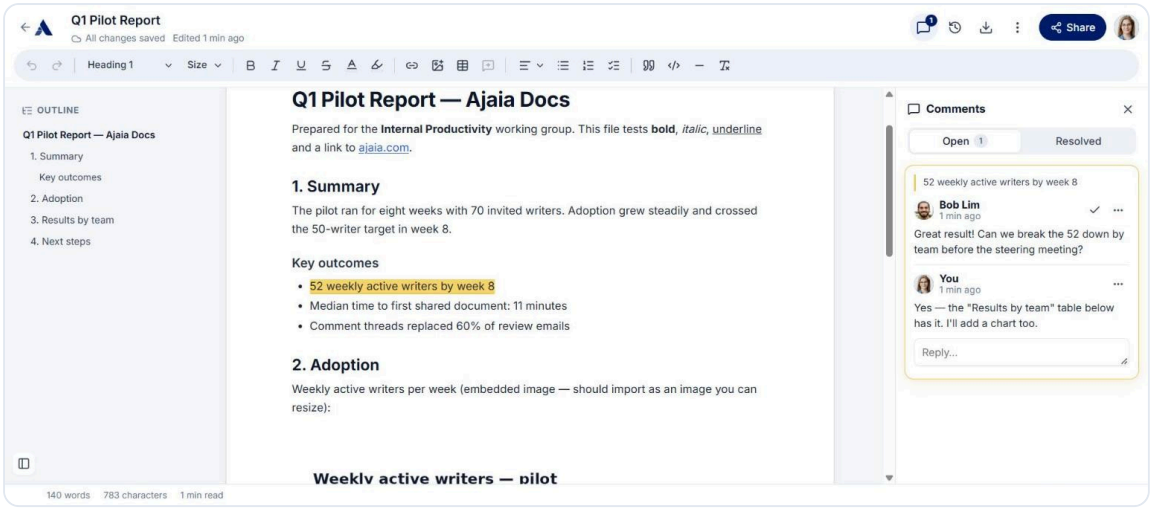
S4 · Embedded image + table from the .docx — Word images become stored, access-checked assets; images are resizable



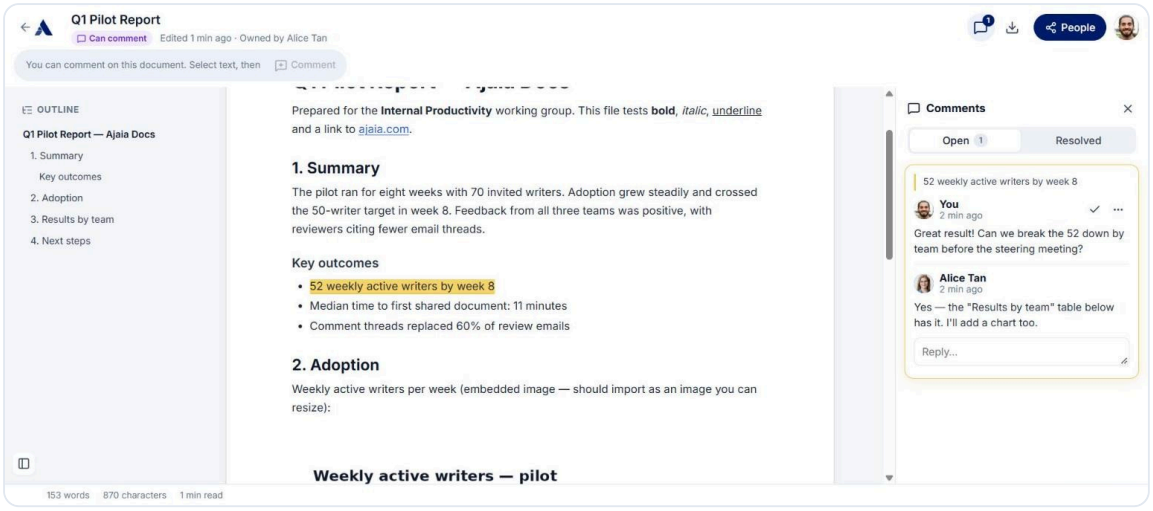
S5 · '/' command menu — insert any block from the keyboard



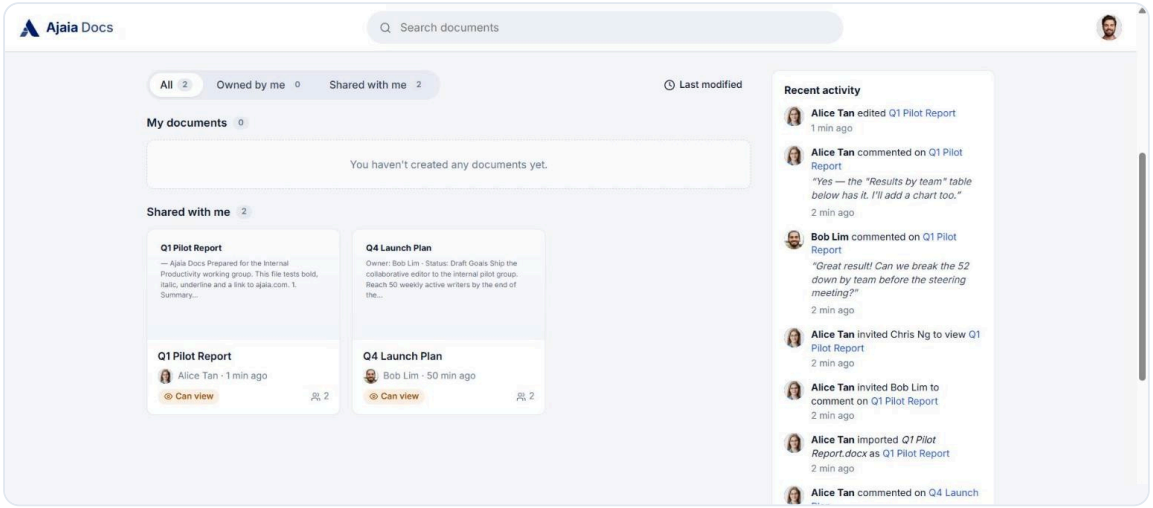
S6 · Sharing — the owner grants Editor / Commenter / Viewer by email, and can change or revoke access



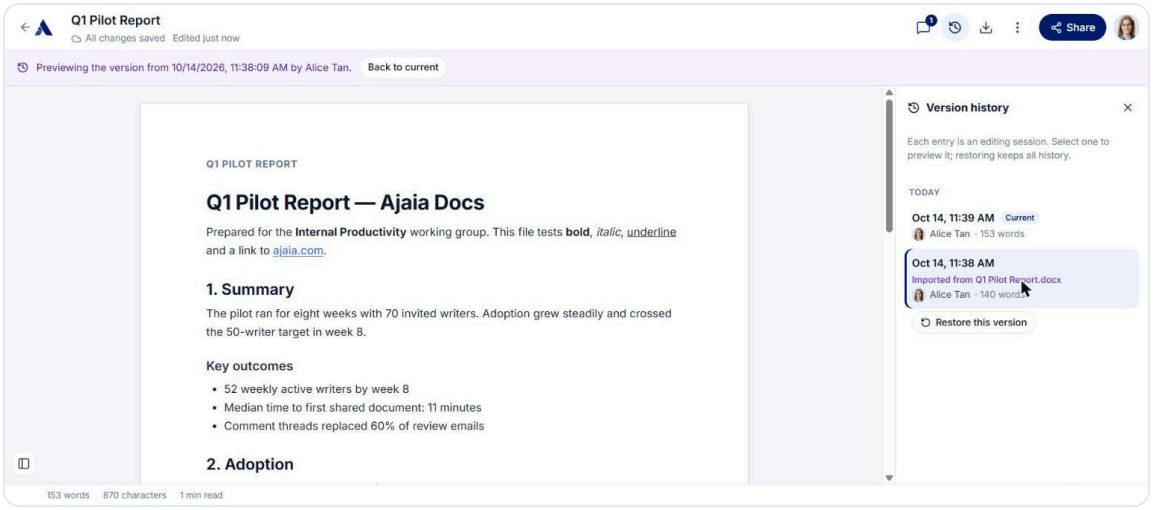
S7 · Comments (owner view) — threads anchored to the selected text; reply, resolve, reopen



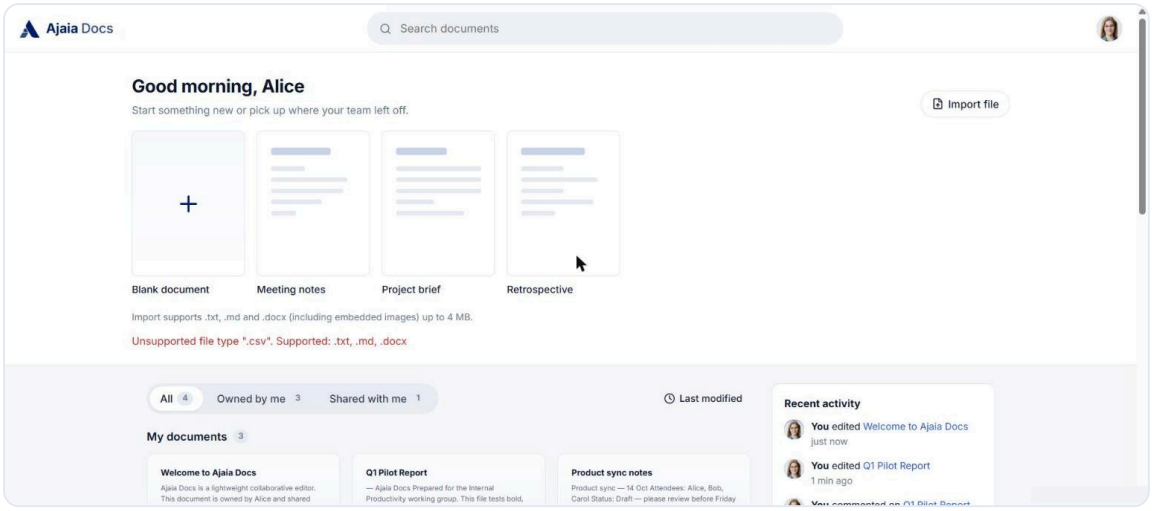
S8 · Commenter view (Bob) — no editing toolbar and a 'Can comment' badge; he can reply to and resolve his own thread. The server rejects edits whatever the UI shows.



S9 · Shared with me (Chris, viewer) — documents from others are labelled with owner and role; the activity feed only covers documents he can access



S10 · Version history — editing sessions by author; previewing the imported version before restoring, which keeps all history



S11 · Validation — an unsupported file is rejected with a clear message; limits are stated next to the Import button

3 · Product judgment: what I prioritised and cut

I treated the product as **“a shared document people can trust”** and went deep where a shared document fails in practice:

- **Who can do what**: one permission matrix (`lib/access.ts`), enforced on every API route, including image bytes. Users without access get 404, so document ids can't be probed.
- **Never losing work**: debounced autosave with retry, optimistic-concurrency conflict detection (409 → *Load theirs / Keep mine*), and version history where even a restore can be undone.
- **Review without editing**: a commenter role and comments anchored by quoted text, so commenting never touches the document body.
- **Files that keep their structure**: imports produce real headings, lists and tables; images inside Word files become stored assets.

Deliberately not built

CUT	WHY	NEXT STEP
Real authentication	Adds nothing new for this brief; authorisation is the interesting part, and it is real.	Auth.js magic links; only `auth.ts` changes.
Character-level co-editing (CRDT)	Needs Yjs plus a socket server that Vercel can't host. Conflict checks, 4 s sync and presence are safe and visible instead.	Yjs + Hocuspocus on a small server.
Invites for people without an account	Needs email delivery.	A pending-invites table claimed on first sign-in.
Suggestion mode	A large feature on its own; comments cover review.	Insert/delete marks with accept/reject.

4 · Architecture in one page

Stack

- Next.js 16 (App Router) + React 19 + TypeScript strict
- TipTap 3 / ProseMirror editor
- libSQL: SQLite locally, Turso in production; no ORM
- Vercel + Turso free tiers, so reviewers pay nothing

Key decisions

- **Versions = editing sessions**, not every save; restoring adds a version.
- **Comments as decorations**, not marks, so commenters need no edit rights and comments cause no conflicts.
- **One `/sync` heartbeat** gives presence, change detection and comment refresh without WebSockets.
- **Sanitise on write**: an allow-list mirrors the editor schema; images only from the app's own asset endpoint, checked by magic bytes, no SVG.
- **Migrations** upgraded the live database in place, with a test that upgrades a v1-schema database.

Data model

documents	owner, title, HTML, version
document_shares	user, role (viewer / commenter / editor)
document_versions	session snapshots + author + label
comments	thread, quote + prefix anchor, resolved
assets	image bytes, checked mime
presence · activity	heartbeats · feed

Quality

- **31** Vitest tests on the domain layer (fresh in-memory DB each)
- **6** Playwright tests with 2–3 simultaneous users against the production build
- Strict `tsc`, clean `next build`, server-side validation everywhere

5 · How AI was used, and where I overruled it

I worked with Claude (Cowork) as a pair-programmer with a shell and a real browser. The process was: fix scope and constraints first; build and test the domain layer before any UI; then *generate → run → look → correct* with type-checks, unit tests, Playwright runs and screenshot review at desktop and phone widths.

> **Hands-on QA, side by side.** I tested every flow on the live deployment myself, with two browser windows signed in as different users: owner next to commenter or viewer. I checked sharing, comments, presence, live updates and permissions from both sides at once, and imported the sample files to compare against the originals. What I found went back into the loop. One example: reviewing the live screenshots showed that imported Word files with images got an empty first history version. That was fixed with a regression test and redeployed before this report.

AI DEFAULT / FIRST DRAFT	WHAT I DID INSTEAD
Store comments as marks in the document HTML	Anchored by quote + context and drawn as decorations, so commenters don't need edit rights.
Snapshot a version on every save	Session-based versions; restore is non-destructive.
WebSockets for presence	Not possible on Vercel; one batched heartbeat endpoint instead.
Trust the browser's MIME type for uploads	Magic-byte checks; SVG refused; access check on every image request.
`ALTER TABLE` to add the new role	SQLite can't alter a CHECK constraint, so I wrote a migration runner and a table rebuild, plus a test.
First deploy returned 500s	Diagnosed from the browser: a dependency needed Node ≥ 22.12, so I pinned the runtime.
Opening a document triggered a save	Found in screenshot review (a 403 in the console for a commenter) and fixed.
Import code dropped Word underline and Markdown strikethrough	Caught by running the shipped sample files through the real import path; fixed and covered by a test.
AI-generated faces for demo users	Used real photos I supplied, cropped to 256 px, with an initials fallback. Sample charts are drawn from exact data, not generated.

The full note is in `AI_WORKFLOW.md`.

6 · Status

Working end to end

- All core requirements (section 1)
- Roles, comments, version history, presence, export
- Word / Markdown / text import with images
- Live deployment with seeded demo data

Next 2–4 hours

1. Yjs co-editing on a small WebSocket service, keeping HTML snapshots for history and export
2. Suggestion mode on top of the comment-anchoring model
3. Real sign-in (magic links) and invites for people without accounts

Deliverables in the Drive folder

`README.md` · `ARCHITECTURE.md` · `AI_WORKFLOW.md` · `SUBMISSION.md` · `SUBMISSION_REPORT.pdf` · `VIDEO_URL.txt` · `docs/screenshots/` · `sample-files/` · full source with tests.